

README for esp32 presence status display

Contents

README for esp32 presence status display.....	1
Project overview	2
Aim of the project	2
System and networking architecture	2
Hardware list and circuit diagram.....	3
Software architecture – python and ESP32	5
Main.py	5
Calendar_logic.py	5
Calendar_UX.py	6
Json_storage.py	7
Calendar_network_server.py	7
MAIN_CODE_FOR_SENDER.cpp.....	8
MAIN_CODE_FOR_RECIEVER.cpp	8
Design choices and trade offs	10
Hardware and pin mapping.....	10
System architecture and communication.....	10
Data serialization and memory safety	11
Receiver and display management	11
Python calendar backend.....	12
Calendar validation logic	12
Problems encountered and their solutions	13
Limitations and future improvements	13
Figures/images.....	14
Setup, instructions and troubleshooting.....	14

Project overview

This project consists of two parts. My own custom calendar python application as well as two ESP32's that each act as a sender and a receiver respectively. The user interacts with a modular python terminal interface to manage their daily activities. The python code folder manages all of the core logic, like checking for overlaps, error handling, storing the user data via JSON. When the user exit and saves they can run the networking program which transmits their existing Json data over a local Wi-Fi network via a HTTP POST request to the sender ESP32. From there it uses the ESP-NOW protocol to wirelessly transmit the same data to the ESP32 receiver. Lastly the data received by the second ESP32 is displayed on an Adafruit st7735 TFT screen which acts as a live presence status display.

Aim of the project

The actual aim of the project was to simply alert people outside of my bedroom door that I was busy by showing what is on my calendar. But of course, this is not a simple task and hence the hidden aim was to create a robust and modular calendar app and then send that data across a network to be parsed by a microcontroller. I separated all logic in the python folder. From UI, to validation logic, to data storage etc into different python files and then using dependencies. Now I vaguely had objective which I wanted to be fulfilled before starting the project. These were:

- Being able to add, edit and remove activities for a given day inside of the terminal without crashing due to incorrect input (either logical or erroneous data)
- Being able to validate all data entered inside of the python program with a variety of algorithms. E.g. preventing time overlaps
- Format the python date objects in some form and store it
- Get said formatted python data and send it over Wi-Fi via an endpoint
- Reliably transmit the Json packet via ESP-NOW
- Parse the JSON on the receiver ESP32 and display what I am doing on a TFT display hooked up to the receiver
- Use non-blocking timers to run an idle animation when the schedule is empty
- Handle network drops using try/except blocks.

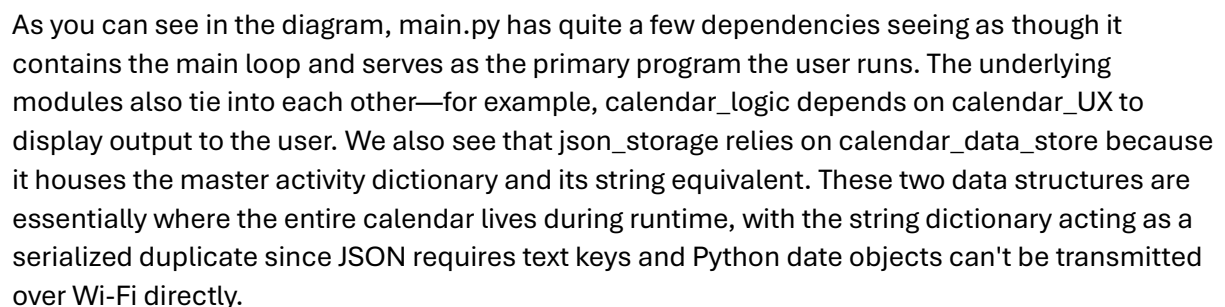
System and networking architecture

Please see the high level diagram for the high level networking pipeline for my application:



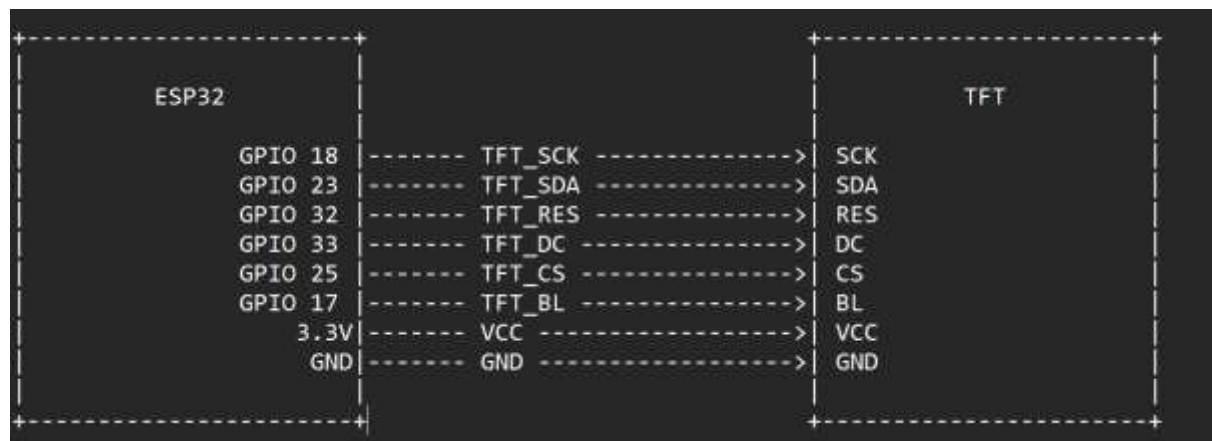
As you can see above, the data is created by the python client. Specifically inside of the main.py file. Then the user runs the python file calendar_network_server.py. This is the second block, where the HTTP POST request is pushed to the ESP32. In this block, we see that the packet is

as for the python program, this can also be shown separately with a diagram since it is a large part of the entire system. Please see the diagram of the python dependency architecture below:



For this project minimal hardware was used, please see the short list below:

- As for the circuit diagram, there is technically only one circuit which is just the TFT display wired directly to the pins on the EPS32. Here is the ASCII schematic bellow:



Software architecture – python and ESP32

For the breakdown of the software architecture, I will go through each python file as well as the sender and receiver ESP32 code one by one. I will start with python:

Main.py

This is the main entry point where the entire application runs. I use a while loop that keeps running the program until the user chooses to terminate by entering 4. When the user starts the program, it loads any existing calendar data from the local JSON storage file called `json_storage.py`. It converts the stored string JSON data into actual python date objects so the program can access and manipulate specific fields. Inside of the loop the user's choice for months is managed. That being a custom month or the current month. This gets handed over to `calendar_logic.py` via `handle_day()` which manages all the actions you can do on a day those being adding editing or removing. When the user leaves the loop, the function `convert_dictkey_to_string()` converts the date key dictionary back into a string dictionary and then finally I call `save_calendar()` to persist the calendar data to a JSON file.

Calendar_logic.py

This is the largest python file. It contains all the core logic for manipulating days, processing the user's input, and managing activities. It mostly contains functions which each conduct a specific action. I'll give a list of these functions and a short description of what they do:

- `get_current_date_information()`, this function gets the current date information by simply returning the current year, month, day and grid object
- `edit_activity_name()`, this loops through all of the activities in a day, then asks the user to pick a specific item. Then updates the name
- `edit_activity_time()`, this does the same excepts handles updating activities start and end times while validating the inputs
- `edit_activity_category()`, this lists the categories for the day and lets the user choose one to change
- `Edit_Activity()`, this is the main function that reads the edit menu choice and based on that choice calls either the edit activity name, time or category function.
- `Add_Activity()`, this formats the new activity data into a temporary dictionary and then appends said temporary dictionary to the correct date inside of the master activity dictionary. Creating a new list if the date does not already exist yet.
- `Get_Activity_Information()`, this simply prompts the user for the name, start time, end time and the category for a new activity.
- `remove_activity()`, remove activity lets the user select an activity and its relevant information and deletes it from the calendar.
- `does_a_given_day_have_activities()`, this function simply returns a Boolean value based on if a day has activities or not
- `get_user_menu_choice_for_Day()`, this checks if a day has activities by using `does_a_given_day_have_activities()` and then displays either the full calendar options or displays a default empty day menu

- `handle_add_activity()`, this uses error handling for when the user wants to add an activity. It mainly uses said error handling to make sure times are valid and collision free.
- `handle_remove_activity()`, similarly this function uses error handling for removing an activity. Simply for erroneous data (i.e. the user not entering a number when asked what activity to remove)
- `handle_remove_activity()`, again the same here. I throw an exception when the user does not enter an integer
- `handle_main_month_choice()`, this uses a while loop and error handling to force the user to enter either 1 or 2. For the current or custom month respectively.
- `handle_user_choosing_day_of_month()`, this does the same except prints the month calendar and loops until the user inputs a valid integer day
- `handle_user_choosing_custom_month()`, this displays the full year calendar and then asks the user to enter a month 1-12. It validates the range and returns the choice.
- `handle_day()`, this function controls the flow of the program for a specific date. I get said date via `get_user_menu_choice_for_day` and then with that choice call either add, remove or edit activity. Otherwise I exit the program (by returning true which I then use in `main.py`)

Calendar_UX.py

Again, somewhat of a large file, this one is purely responsible for terminal presentation and visual formatting. I use the rich library to make the regular plain text I would otherwise be using appear nicer with colour, fonts, and styling. It also allows me to use panels and structured tables, which overall turns the user experience from looking like a raw developer script into something that feels like an actual end-user application. `Calendar_validation.py`. here are a list of all the functions and briefly what they do:

- `show_success()`, this is a custom function solely for printing a styled green panel with a checkmark to confirm that the user did a successful action like adding an event
- `show_warning()`, this does the opposite. It prints a bold red panel with a warning icon to alert that the user did an incorrect action.
- `show_prompt()`, a custom function which prints a distinct blue panel for inputs required from the user
- `display_day()`, this checks if a day has activities, if it is empty it prints a notice in yellow, otherwise it creates and outputs a structured rich table including the index, activity name, start time, end time and the category for that date.
- `show_empty_day()`, uses a yellow panel letting the user know that there are no activities for that day
- `Existing_Times_Of_Activity()`, builds a red bordered table showing activities that conflict with each other and their time slots inside a warning panel when the clash happens..

- `show_day_menu()`, if a day has activities show available actions for a day. (add, remove, edit for a day with activities, or add activities for an empty day)
- `show_edit_menu()`, renders the edit options panel and then uses a while true loop with error handling to make sure the user inputs a valid integer between 1 and 3 (for the edit options name time or category)
- `show_calendar_information()`, prints a cyan panel to display the calendar to the user
- `show_calendar_information()`, and finally this function shows the full action menu for a day with activities and uses error handling to force the user to enter a number 1-4.

Json_storage.py

This is the persistence file. This file is the bridge between the master activity dictionary and Json. It converts said dictionary to and from strings so it can be saved and loaded from the user's disk. Again, this file also contains several smaller functions, they are:

- `load_existing_data()`, as suggested by the name this function loads the existing data inside of the `calendar_data.json` file. It does this by opening it and then using the load function. It then returns the master activity dictionary string
- `save_calendar()`, save calendar does the opposite. Here I dump the Json calendar dictionary string into the same file. Writing (writing is key rather than appending)
- `convert_string_to_date()`, this converts the master activity string dictionary back into the python type (i.e the master activity dictionary where I can access and manipulate the date fields). This is important because as stated before I have designed the program to have these two separate states and then convert between them.
- `convert_dictkey_to_string()`, this does the opposite. I convert the python type back into a string for when I want to save the file. (since the save calendar function works with the string master activity dictionary)

Calendar_network_server.py

This file runs as an independent indefinite loop. It bridges the gap between the python environment and the ESP32's. it continuously gets the latest master activity dictionary by loading existing data then formats said calendar into a Json string and lastly uses requests to send the Json payload over. It prints the response. Text for debugging purposes too. In this file I also use error handling to handle timeouts or dropped connections. There are no functions in this file. Just the loop. So, I won't break that down.

Then we have the ESP32 sender and receiver code respectively. For these I will go through setup and loop as well as functions/code outside these scopes.

MAIN_CODE_FOR_SENDER.cpp

In the sender I start with the dependencies. Which are the esp_now library, Wi-Fi and the web server library. After this I define the port, we use. Which is 80, as well as create the web server object. I also define the SSID, password and receiver mac address. And then I define the first function.

- `handleData()`, this is the core function that handles incoming HTTP requests on the sender ESP32. When the Python program sends a POST request to the `/data` endpoint, this function gets the payload, logs it to the serial monitor for debugging, and then bridges it over to the receiver. At the top of the function, I extract the raw HTTP request body as a string using `"plain"`. Because Python sends the data as a string formatted as raw JSON, `server.arg("plain")` ignores any standard HTML fields or URL parameters and pulls out that exact raw text string. Once it's received, I print it out to the terminal as diagnostic code so I can track what's coming in. From there, we use the `esp_now_send()` function to beam that JSON body straight to the receiver using its MAC address. Because the low-level ESP-NOW radio layer only accepts raw bytes rather than high-level strings, it casts the body into a pointer of unsigned 8-bit integers (`uint8_t*`). I also calculate the precise byte size of the data, adding 1 to ensure the null-terminating character is included. Finally, it checks if the radio transmission returns `ESP_OK` to log a success or failure and shoots an HTTP 200 response back to Python to complete the loop.
- `senderCallback()`, this is an asynchronous callback function on the sender ESP32 that automatically gets called when an `esp_now` transmission finishes. `Esp_now` does this automatically, you need to pass the mac address of the receiver to this function as well as the status of the message which has a specific type being `esp_now_send_Status_t`. this simply tells us if it was successful or not, which we use inside of the function to print to serial either "packet delivered" or "packet failed".
- `Setup()`, this is the function that runs once for any Arduino/ESP32 program. Here at a high level, I initialise serial, create Wi-Fi connection, configure HTTP, initialize ESP_NOW radio, register callback and setup the receiver as a peer.
- `Loop()`, and here all I do is call the function `handle client`. This constantly checks for incoming network traffic. It handles client connections, parses http requests and then triggers the `handleData()` handler.

MAIN_CODE_FOR_RECIEVER.cpp

As for the receiver, I start with the dependecncies. This is the Adafruit GFX and ST7735 libraries for the TFT display, SPI, ESP_NOW, Wi-Fi, ArduinoJson and finally `time.h`. `ArduinoJson` of course is so that we can parse the data, and `time.h` is needed to get the real time. After the include statements, I define the pins for the TFT screen including the clock line, data line, reset, chip

select and backlight. I also create a 512-byte buffer which holds the incoming JSON, create the JsonDocument object to store it. Then I define the functions. These functions are:

- `recievercallback()`, this is the callback that automatically triggers when the ESP32 receives an `esp_now` packet from the sender. First, it checks if the incoming data actually fits inside of the buffer so that it doesn't crash. Then it copies the data over and adds a null terminator so it's a valid c++ string. Then the volatile Boolean flag `received` turns into true, so the main loop knows we have new data to parse.
- `timeToMinutes`, this is a helper function which takes the HH:MM string from the json and then converts it into total minutes past midnight which makes time collision math easier.
- `displayActivity()`, display activity simply extracts the name, start time, end time and category, clear the screen to black and then print out the new calendar information in white text.
- `displayNoActivity`, this just displays "no current activity" on a black screen.
- `findCurrentActivity()`, quite a large function which checks what should be displayed on the TFT display at any given time. It uses `getLocalTime()` via NTP which gets the real-world time converts it to minutes then loops through the Json array of activities (defined as `JsonArray activites = calendarDocument.as<JsonArray>()`;) I check if the current time falls inside any of the activity intervals and if it does then I return the index of that activity. Otherwise, I return -1 which is the sentinel value to show that nothing is happening right now.
- `setup()`, this is the function that always runs once upon running the code. In here I initialize the SPI connection and turn on the TFT backlight. Then I connect to Wi-Fi which I needed to configure NTP time synchronization so that the esp32 know the actual time. Then I set the channel to being 6 which matches the sender, initialize `ESP_NOW` and register the receiver callback function.
- `loop()`, this handles the main execution in three parts. First, it checks the received flag. If it is true, it then uses `ArduinoJson` to deserialize the raw text buffer into the `calendarDocument` object. Then I use a non-blocking `millis()` timer to check every 1000ms where it calls `findCurrentActivity()`. Now if the activity has changed since the last check the screen will be updated. 1 second is more than ample. Also if the current activity index is -1 it constantly calls the animation function.
- `initGameOfLife()`, this initializes the grid size of the conways game of life grid. It does this based on the actual width of the TFT screen and height. I then define the cell size as being 4 pixels and then divide the width/height by this value.
- `randomizeGrid()`, this function uses randomly generated numbers to create a starting state of the game with roughly 28% cell density. This is a value enough for no crazy initial fizzles yet keeps going for a while.

- `drawFullGrid()` this renders the game by iterating through rows/cols then calling the draw function
- `countAliveNeighbours()`, this function checks through its surrounding 8 pixels and then increments a counter. It also considers wrapping of the grid by using modulo
- `stepGameOfLife()`, this calculates the next generation based on the rules of the game (cells survive with 2 or 3 neighbours and dead cells come alive with 3 neighbours). This only updates the specific pixels on the TFT when they change state. Which improves performance. It also tracks staleGenerations and generationCount to start the simulation again incase oscillators populate the screen and there is no real progression in the game.
- `drawGameOfLife()`, this function has all the animation logic inside of it. It has its own non-blocking timer separate to the loop since it needs to have a suitable frame rate. 1000ms inside of the loop would be too slow and hence I chose a `life_frame_delay` value of 150ms.

Design choices and trade offs

With this architecture for this application. There were several technical choices made for reasons like robustness, responsiveness and maintainability. Ill separate these in different subsections. Please see below:

Hardware and pin mapping

At a high level. Hardware setup is simply getting the two ESP32's communicating with each other. As well as this I made obvious but crucial decisions on what pins to use on the receiver ESP32..

- I used the dedicated SPI hardware pins. I mapped the TFT display clock and data lines to the dedicated hardware SPI pins on my ESP32 as opposed to random GPIO pins. This made sure I had high speed bus timing for screen drawing/rendering.
- I also made sure to avoid GPIO TX0 and RX0 for chip select/control lines to avoid corrupting the serial line.

System architecture and communication

- I split the firmware into two independent PlatformIO projects that being sender and receiver. This isolated the build environment and made developing significantly easier. It helped massively in the debugging process since I isolated errors. If an issue occurred in the sender this does not affect the way the receiver behaves.
- Layered responsibility was used. By this I mean the python files own the data. They store the data and manipulate it. The sender is purely a dumb transport bridge, and then the receiver handles time logic and rendering. This was a very good design choice for several reasons. First, it meant that there was not as much logic duplication, secondly it meant

changing one layer e.g. updating the calendar storage format does not cascade into the microcontroller structure and cause that to break. Another reason is resource management. ESP32s do not have much RAM / processing power and hence doing the python work on the desktop computer and then keeping the sender as a lightweight radio/wifi bridge and then giving the receiver the responsibility of state evaluation and TFT management prevented both boards from being overloaded with tasks. This makes the system very responsive. And lastly separation of responsibilities. Decoupling the architecture meant I could easily isolate errors to their specific layer. One example of this would be if I saw debugging messages that suggested Wi-Fi, esp_now, and Json data were all fine but nothing was being displayed on TFT. I could think receiver owns TFT → check receiver → check TFT rendering.

- I used ESP_NOW instead of purely Wi-Fi or MQTT since it does not depend on a router. It is very low latency and has long range. It also acted as a learning curve. Transmitted pure bytes over a separate protocol was very valuable in this project. It forced me to think at a lower level. It also makes sense from a systems perspective since if I wanted several calendars/screens/applications that displayed different things I could do this. E.g. a kitchen display for a cat feeding schedule, a living room schedule for who's currently home. I could simply make the sender transmit different payloads to different receivers. The displays can all remain dumb and just receive bytes and then display data.

Data serialization and memory safety

- I formatted raw JSON strings byte for byte via ESP_NOW rather than using a binary protocol. It's slightly less bandwidth efficient but it's a uniform format which is created in the python code and displayed right to the TFT display. This simplified debugging massively. It also led to a clean structure of serializing and deserializing. And a clean solution to persistence in python (using master_activity_dictionary_string to write to a Json file since it's the string representation of the python calendar) I made the receiving char array quite large, 512 bytes specifically. And forced null termination (via buffer[length] = '\0'). This prevented overflow errors or incorrect Json parsing due to memory limits.

Receiver and display management

- I kept the callback for the receiver purposefully light. I made it so that it simply copies payload bytes and then raises a volatile flag. Which meant all the heavy tasks like JSON parsing, time checking and TFT rendering was part of the main loop. Again, separating responsibilities and keeping each function modular. Good programming practice and helped massively with debugging.
- I used a non-blocking timer inside of the loop for receiver to constantly evaluate the calendar states independently of incoming network packets. This meant that the display would update almost instantly when an activity is added/starts.
- Lastly, I used currentIndex and displayedActivityIndex to cause screen updates based on state transitions. This prevented unnecessary drawing which saves computation and makes the screen less flickery. This was done via sentinel values (-2 for uninitialized vs -1 for no current activities).

Python calendar backend

The python backend is heavily reliant on one data structure. And that is the `master_activity_dictionary`. This is a dictionary where keys are native date objects, the values are lists of activity dictionaries. So, the mapping is a dictionary → list → dictionary. This allows a single date to hold multiple activities. which fits the function of the calendar. This is not the only data structure; I also created the `master_activity_dictionary_string` which is the string version of the original data structure. During runtime `master_activity_dictionary` is manipulated and changed. And when the program ends by the user entering 4, `master_activity_dictionary_string` stores all of the new updated calendar data inside of it as a string. To do this I needed boundary functions like `convert_string_to_date` and `convert_dictkey_to_string`. This keeps serialisation separate from the `master_activity_dictionary` data structure during runtime. By this I mean persistence is only dealt with upon loading the program and saving the calendar. This is technically a disadvantage of the program since if the program crashes during runtime the updated calendar data the user adds is lost. `Main.py` is where this loading/saving happens since it imports the functions from the other files (`json_storage.py` etc). I separated the responsibilities across several files with my python backend. each file has different functions associated with the overall goal of that file which you can see above. One non ideal design architecture decision was to pass `master_activity_dictionary` as an explicit parameter and then mutate it in place. This does not create a single source of truth since in `main.py` the import from `calendar_data_store` gets shadowed when reassigned via the conversion function. Moving onto the UI, I made sure to use counting starting from 1 for when the user wants to choose what activity on a day to edit for example. This is translated to a 0 based list inside of the code throughout the program which was deliberate. As for validation, `validate_activity` uses a parameter `editing_index`. This is -1 by default when adding but when editing the actual index is passed. This excludes an activity from being checked against itself when editing an activity. And lastly, I intentionally kept time in the format `hh:mm` rather than `datetime.time` objects. this works since I have my own separate `convert_time_to_minutes` function which converts said format to minutes which allows for easy comparison within the program.

Calendar validation logic

In this section I'll discuss several parts of the program which ensure the program does not break or not behave as intended. Specifically, how time is defined and verified, how clashes are dealt with, as well as error handling during any sort of user input.

Time format and syntax checks – before any activity is processed, I use `validate_time()` to make sure start, and end times are in the correct form. If a user enters incorrect data i.e `aaaaa:rrrrr` this will cause the user to be prompted again. The validation layer of the program does this for every type of error. Whether this be a range issue or an erroneous data issue. This also means the program does not need to continue and even attempt to check for clashes when incorrect data is entered. As stated, before I use raw minutes rather than using the time in the `hh:mm` format. This makes the program very straightforward since all I need to do then is simple inequality checks. The core logic is two activities overlap unless one finishes before the other

starts. Which covers all cases. `Validate_activity()` deals with this. When the user enters any data, and by that, I mean any. Whether it's a new name for an exiting activity, or a number to correspond to what activity to remove in a day. These are all wrapped in a robust while true loop with a try/except ValueError block. I name these handle {insert function name here} since they handle a functions input for them. this always catches invalid inputs and outputs a warning from the UX file, and then re prompts the user.

Problems encountered and their solutions

I encountered several issues while creating this project. Read the following (they are in no particular order)

- Initially I had confusion with what the different pins on the ESP32 even allow in the first place. I mistook SPI requirements for I²C conventions.
- I was considering routing the display pins to GPIO 1 and 3 which are boot diagnostic pins which would not work
- I initially used a singular LED on the receiver ESP32 to prove I could even send a singular byte representing a Boolean value in the first place. I had issues with this like not knowing how to send data or how to represent the Boolean value but later I learnt.
- Since I'm using two identical ESP32's with two identical cables I had mixed them up and hence checked device manager to confirm which one was which (sender on COM6 while receiver was on COM8)
- I had issues with unreadable terminal output which was a mismatch between the baud rate of serial and the baud rate set inside of platform io configuration.
- I did not set the peer channels equal to each other, this was because the WiFi radios of each peer were on separate frequency bands. So, I forced both to channel 6 via `esp_wifi_set_channel()` and `peer.channel`
- I was confused on how callbacks worked seeing as its quite ambiguous to when they're called and why they even need to be defined in the first place.
- I also learnt how to format the `esp_now_peer_info_t` struct via clearing memory with `memset()`. Before I had done this the struct sometimes contained previous obsolete data.
- I had a bug with the sentinel value of -1. Specifically, both `currentActivityIndex` and `displayedActivityIndex` defaulted to the value of -1. This tricked the program into thinking there hadn't been a change in activity, so I set `displayedActivityIndex` to -2 which show no current activity upon booting the TFT screen.
- Early on I was not sure on when to save the users calendar data, I later decided to make this occur when the user explicitly enters the number 4 to manually end the programs loop. This is not the most convenient or user friendly, but it works.
- I had issues with the Json file on whether to append/write to the Json file.
- Originally my `remove_activity()` function removed entire dates instead of specific target activities inside of the dates list.

Limitations and future improvements

My code can be improved massively. Not just structurally but there's also quite a few quality-of-life changes that can and will eventually be made. There are some poor design decisions that

coincidentally work with my program since the code allows it to. Again, this will be improved in the future. Please read below (again in no significant order):

- The python code currently constantly sends HTTP post requests. Even if the calendar has had no changes to it. This is inside of the separate network server code. This is inefficient since I don't constantly need to send the POST requests. They only realistically need to be sent when data has changed inside of the calendar.
- The entire program can be improved massively with OOP. I could rework the model so that I have a calendar, activity, day class etc. it would make the code a lot less repeated and make it significantly more readable.
- While I do still store the state of the last updated calendar in ESP32 memory. If Wi-Fi fails, then the entire system turns ugly. The activities briefly stop displaying. I could improve this in the future
- Transmitting an entire calendar date activities every single update creates a ton of data overhead. I could send smaller updates like the next activity when that time is reached.
- This entire program is unencrypted. The ESP32 now communication is not safe for real use since there is zero security. Again, for the functionality of this program it is fine.
- I haven't even finished the category functionality. There is no checks for valid categories yet. Again this is something to implement.
- Right now, updating or removing an activity requires manually resetting the ESP32 boards, which is far from ideal.
- Relying on breadboards and loose jumper wires isn't great for long-term use. I want to eventually design and order a custom PCB to surface-mount the ESP32s and the TFT screen, turning this from a desk experiment into a solid, standalone piece of hardware.

Figures/images

Setup, instructions and troubleshooting

Hardware & Requirements

- Get two esp32 dev boards
- Label one as the sender and one for receiver to avoid confusion and to not mix up com ports
- Get a ST7735 SPI TFT Adafruit display
- Jumper wires and breadboard
- The mapping for the receiver is:

Receiver Board → TFT Pin Mapping		
TFT Pin	ESP32 GPIO	Notes
SCK	GPIO 18	Dedicated hardware SPI clock
SDA / MOSI	GPIO 23	Dedicated hardware SPI data out
RES	GPIO 32	Reset pin
DC	GPIO 33	Data/Command pin
CS	GPIO 25	Chip Select pin
BL (Backlight)	GPIO 17	Driven HIGH for screen illumination
VCC	3.3V	Power supply
GND	GND	Ground

- Source = Gemini

Software & Environment Setup

- Download the relevant esp32 dev board platform / code functionality in your IDE of choice
- Set up platformIO in two separate projects on separate windows.
- Install the following Arduino libraries : Adafruit_GFX, Adafruit_ST7735, SPI, WiFi.h, WebServer.h, esp_now.h, ArduinoJson.
- Use a python IDE of your choice with required networking package. (pip install requests). Make sure all python source files are downloaded in the same local project directory.

Configuration

- Flash each esp32 once, open its individual serial monitor and note its mac address.
- Input the receiver's mac address array into the sender's firmware.
- Ensure both microcontrollers are on the same Wi-Fi channel.
- Use your own routers SSID and password.
- Do not use configtztime if you are outside the UK / GMT region.

Instructions & Usage

- Power on both esp32s and always check the serial monitor to confirm that they have successfully connected to Wi-Fi and NTP time sync has finished.
- Run main.py in your python environment to manage your calendar.
- When you have quit main.py via entering 4, launch calendar_network_server.py and run the program.
- Verify on the sender's serial monitor that the packet was forwarded to the receiver. It should say "new calendar data received". The calendar data is also shown in python's terminal.

Troubleshooting

- If you get errors such as (removing unity @2.6.1), simply reset or reinstall platformio
- If serial looks like garbage, make sure there is no bad rate mismatch between serial.begin and in platform io settings.

Updating or Removing Activities

- To edit or remove an activity from the calendar (rather than just adding one), and the receiver TFT screen does not automatically update. You must do the following to re-sync the system:
 1. Reset both ESP32 boards.
 2. Run the main Python calendar program.
 3. Exit the main program by selecting option `4`.
 4. Run the Python network server program.
 5. Open the Serial Monitor on the Sender ESP32.
 6. You should now see the updated activity.